

ATOM Personal AI — User Manual & Presentation Guide

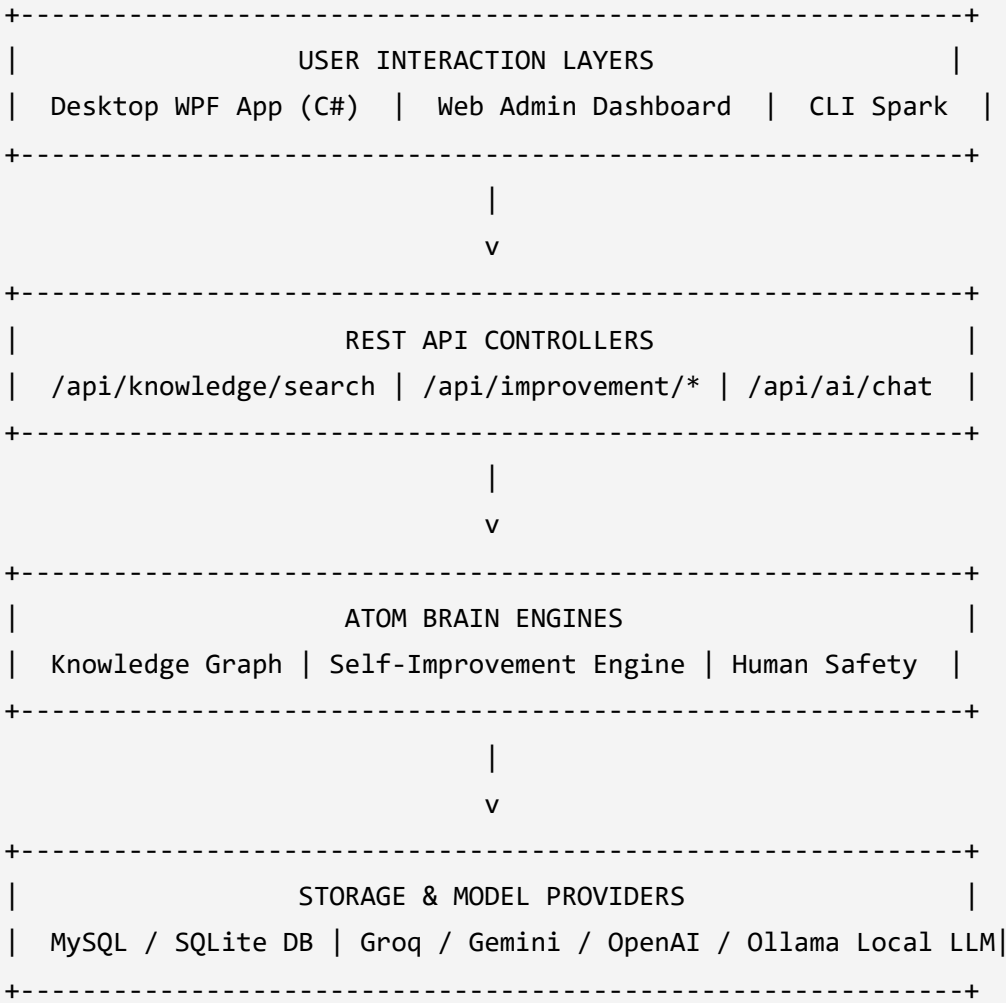
Advanced AI Knowledge, Memory, Self-Learning, and Human-Guarded Self-Improvement Engine for ATOM

Table of Contents

1. Overview & Architecture
2. Quick Start & Application Launch
3. AI Provider Configuration (OpenAI, Gemini, Groq & Local Ollama)
4. PDF Ingestion & Document RAG Training
5. Core Capabilities & Knowledge System
6. Self-Learning & Self-Improvement Engine
7. Desktop WPF Assistant Application
8. CLI Spark Commands Reference
9. REST API Endpoints Guide
10. Presentation Deck Slides (PPT Outline)

1. Overview & Architecture

ATOM is a state-of-the-art hybrid Personal AI Assistant built on PHP CodeIgniter 4 backend services, MySQL/SQLite database persistence, and a modern C# .NET 9 WPF desktop client.



2. Quick Start & Application Launch

Launch all ATOM services (Backend API, CLI Evaluation, and Desktop WPF Client) in a single command:

Option 1: Run `start-all.bat`

```
.\start-all.bat
```

Option 2: Run `atom.bat` Menu

```
.\atom.bat
```

Select option `[A]` — *Start All Services (Backend + CLI + Desktop UI).*

A. Start Backend API Server:

```
cd backend  
php spark serve --host 0.0.0.0
```

(Backend HTTP API will run at `http://localhost:8080`)

B. Launch Desktop WPF Assistant:

```
dotnet run --project src/PersonalAIAssistant/PersonalAIAssistant.csproj
```

C. Run Self-Learning Evaluation via CLI:

```
cd backend  
php spark atom:self-improve
```

3. AI Provider Configuration (OpenAI, Gemini, Groq & Local Ollama)

ATOM supports 4 primary LLM providers. Credentials and model selections are configured in the `.env` file located in the workspace root.

1. Obtain an API key from platform.openai.com.

2. Edit `.env`:

```
LLM_PROVIDER=openai
LLM_MODEL=gpt-4o-mini
LLM_API_KEY=sk-your-openai-api-key-here
LLM_API_URL=https://api.openai.com/v1
```

1. Obtain an API key from [Google AI Studio](https://aistudio.google.com).

2. Edit `.env`:

```
LLM_PROVIDER=gemini
LLM_MODEL=gemini-3.6-flash
LLM_API_KEY=your-gemini-api-key-here
LLM_API_URL=https://generativelanguage.googleapis.com/v1beta
```

1. Obtain a key from console.groq.com.

2. Edit `.env`:

```
LLM_PROVIDER=groq
LLM_MODEL=openai/gpt-oss-120b
LLM_API_KEY=gsk_your-groq-key-here
LLM_API_URL=https://api.groq.com/openai/v1
```

1. Install Ollama locally and pull a model: `ollama pull llama3.1`

2. Edit `.env`:

```
LLM_PROVIDER=local
LLM_MODEL=llama3.1
```

```
LLM_API_KEY=  
LLM_API_URL=http://localhost:11434/v1
```

On Windows XAMPP environments without `cacert.pem`, cURL may report `unable to get local issuer certificate (20)`. Add this line to `.env` to disable SSL verification for local dev:

```
ATOM_DISABLE_SSL_VERIFY=true
```

4. PDF Ingestion & Document RAG Training

ATOM converts PDF manuals and text documentation into indexed vector embeddings and Knowledge Graph triples.

1. Launch Desktop App via `start-all.bat` or `dotnet run`.
2. Navigate to **Knowledge Base Page**.
3. Click **Upload Document** and select your `.pdf` file.

1. Open `advanced-ui/index.html` in your browser.
2. Navigate to the **Document RAG Training** tab.
3. Drag & drop your PDF file onto the dropzone.

```
curl -X POST http://localhost:8080/api/knowledge/upload \  
-F "document=@/path/to/manual.pdf" \  

```

```
-H "Authorization: Bearer your-jwt-token"
```

```
[Uploaded PDF] ---> [PdfExtractor.php] ---> [Semantic Chunker (512 tokens)]  
                                     |  
                                     v  
[Knowledge Graph Triples] <--- [Embedding Engine] <--- [SQLite Vector DB]
```

5. Core Capabilities & Knowledge System

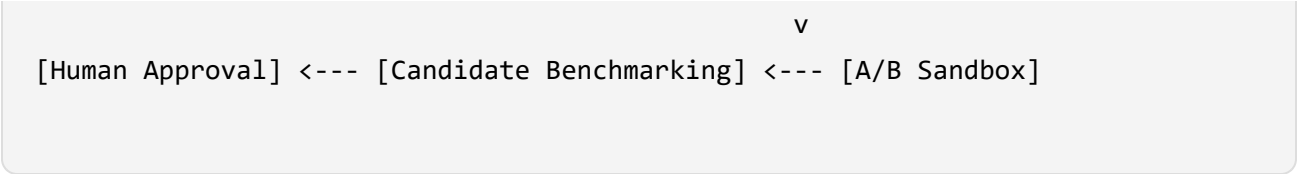
ATOM automatically extracts structured facts into `atom_knowledge_triples`:

- `ATOM -> DEPENDS_ON -> SQLITE_DATABASE`
- `USER_PREFERENCE -> PREFERS -> DARK_MODE`

ATOM queries both Document Chunks AND Knowledge Graph Triples simultaneously via `GET /api/knowledge/search?q=query` to eliminate hallucinations.

6. Self-Learning & Self-Improvement Engine

```
[User Interactions] ---> [Log Metrics] ---> [Flaw Detector]  
                                     |
```



- 1. **Metric Logging:** Interaction metrics logged in `atom_evaluations` .
- 2. **Flaw Detection:** Flags performance flaws if accuracy drops.
- 3. **Sandbox A/B Experiments:** Runs candidate experiments requiring a **+5% improvement**.
- 4. **Human Authorization Gate:** Promoted configs require human sign-off in `atom_human_approvals` .

7. Desktop WPF Assistant Application

The WPF app (`src/PersonalAIAssistant`) features:

- **Chat View:** Multi-turn conversation with LLMs.
- **Knowledge Base Page:** Document management and hybrid search.
- **ATOM Safety Gate Page:** Approve/Reject pending requests and monitor sandbox scores.

8. CLI Spark Commands Reference

Command	Purpose	Example Usage
<code>php spark serve --host 0.0.0.0</code>	Start CodeIgniter 4 backend HTTP server	<code>php spark serve</code>

Command	Purpose	Example Usage
<code>php spark</code> <code>atom:self-improve</code>	Trigger evaluation cycle & A/B sandbox benchmarking	<code>php spark</code> <code>atom:self-improve</code>
<code>php spark</code> <code>atom:approve <id></code>	Authorize candidate experiment promotion	<code>php spark</code> <code>atom:approve 1</code>
<code>php spark migrate</code>	Run database schema migrations	<code>php spark migrate</code>

9. REST API Endpoints Guide

- `GET /api/knowledge/search?q={query}&limit=5`
- `GET /api/knowledge/documents`
- `POST /api/knowledge/upload`

- `GET /api/improvement/flaws`
- `GET /api/improvement/experiments`
- `GET /api/improvement/approvals`
- `POST /api/improvement/approvals/{id}/approve`
- `POST /api/improvement/approvals/{id}/reject`

10. Presentation Deck Slides (PPT Outline)

- **Slide 1: Title Slide** — ATOM Personal AI Assistant (Presenter: VISHNUPRIYAN)
- **Slide 2: Problem & Vision** — Self-improving assistant with persistent Memory & Knowledge Graph.
- **Slide 3: System Architecture** — C# WPF + PHP CodeIgniter 4 + Multi-LLM Providers.
- **Slide 4: Dual RAG Search** — Combining Vector Chunks with SPO Triple Triples.
- **Slide 5: Autonomous Self-Improvement** — Metric logging & A/B sandbox benchmarking (+5% threshold).
- **Slide 6: Human Safety Gate** — Mandatory human sign-off for candidate model promotions.
- **Slide 7: Desktop WPF Experience** — Dedicated UI tabs for Chat, Knowledge Base, and Safety Gate.
- **Slide 8: Verification & QA** — 100% PHPUnit test pass rate & clean .NET solution build.

11. System Prompt & Identity Rules Specification

ATOM's system prompt is configured in `config/rules/system.md`. It establishes 28 core operational directives:

- **User Identity:** Preferred name **Vichu**, profession **PHP / PHP Full-Stack Developer**.
- **Context Isolation:** Topics remain isolated unless explicitly linked by the user.
- **Memory Rules:** Temporary variables and test code are never auto-stored. Permanent memory requires explicit confirmation.
- **Technical Best Practices:** Modern PHP, prepared SQL statements, type declarations, and framework-native solutions.
- **Security & Privacy:** Zero hard-coded credentials, prompt injection defense, and hidden context protection.
- **Core Principle:** *Accuracy over confidence, evidence over assumptions, and clean user-facing errors over raw API tracebacks.*

12. AI Website Builder & Coding Agent Specification

ATOM's software development agent directives are configured in

`config/rules/agent.md`. It establishes 35 core coding rules:

- **Project Root Isolation:** Operations are strictly scoped inside `PROJECT_ROOT` with path-traversal prevention.
- **Targeted Modifications:** Modifies only necessary files to preserve existing working functionality.
- **UI & Code Standards:** Responsive semantic HTML5, clean CSS in `assets/css/`, modular JS, and modern strict-type PHP.
- **Database Security:** Mandatory parameterized queries and environment-based secret management.
- **Verification Workflow:** Executes syntax verification and builds before declaring task success.

13. Coding Agent Control Prompt & Provider Architecture Specification

ATOM's control prompt and AI provider rules are configured in

`config/rules/control_prompt.md`. It establishes 30 operational directives:

- **Provider Consistency:** All AI generation routes through `AIProviderManager`. The UI-reported provider strictly matches actual AI execution.
- **Raw Exception Interception:** Intercepts HTTP 429 rate limits and quota errors, displaying clean user warnings while logging traces privately.
- **Command Intent Routing:** Separates natural chat from filesystem tool calls and multi-step project generation.
- **Real Tool Action:** Performs real filesystem operations (`write_file`, `create_file`) instead of merely printing code blocks in chat.

- **Core Principle:** *ATOM thinks with the AI, acts with controlled tools, and verifies the result before claiming success.*